

REGLAS GRAMÁTICALES ASIGNADAS

Yanzon Facundo

Leg. 46103

Tema: 25914_11

Se propone una gramática en notación EBNF que describe un lenguaje específico. Utilizando ANTLR4 con JavaScript, implemente un analizador que procese un archivo de entrada (input.txt) con código fuente escrito en dicho lenguaje.

```
<programa>::= { <declaración> | <función> | <ejecución> }
<declaración>::= "variable" <nombre> ["=" <valor>] ";"
<función>::= "función" <nombre> ["(" <argumentos> ")"] "{"
    { <instrucciones> }
    "}"
<argumentos>::= <variable> ["," <argumentos>]
<instrucciones>::= { <operación_texto> | <impresión> | <retorno> }
<operación_texto>::= <variable> "=" <transformación> "(" <cadena> ")" ";"
<transformación>::= "mayúsculas" | "minúsculas" | "longitud"
    | "invertir" | "reemplazar"
<concatenar>::= <variable> "=" <cadena> "+" <cadena> ";"
<impresión>::= "imprimir" "(" <valor> ")" ";"
<retorno>::= "devolver" <valor> ";"
<valor>::= <texto> | <número> | <variable>
<cadena>::= <texto> | <variable>
<texto>::= "\"" [caracteres] "\""
<nombre>::= [a-zA-Z_] [a-zA-Z0-9_]*
```

// Regla inicial

programa

```
: (declaracion
  | funcion
  | ejecucion
)*
;
```

// Declaraciones

declaracion

```
: 'variable' nombre ('=' valor)? ';'
;
```

// Funciones

funcion

```
: 'funcion' nombre '(' (argumentos? ')')? '{' instrucciones '}'
;
```

argumentos

: nombre (',' argumentos)?
;

// Instrucciones

instrucciones

: (operacion_texto
| concatenar
| impresion
| retorno
)*
;

// Operaciones de texto

operacion_texto

: nombre '=' transformacion '(' cadena ')' ';' ;
;

transformacion

: 'mayusculas'
| 'minusculas'
| 'longitud'
| 'invertir'
| 'reemplazar'
;

// Concatenación

concatenar

: nombre '=' cadena '+' cadena ';' ;
;

// Impresión

impresion

: 'imprimir' '(' valor ')' ';' ;
;

// Retorno

retorno

: 'devolver' valor ';' ;
;

// Valores

valor

: TEXTO
| NUMERO
| nombre
;

```

// Cadenas
cadena
  : TEXTO
  | nombre
  ;

// Nombre de variable o función
nombre
  : ID
  ;

// Ejecución de función
ejecucion
  : nombre '(' argumentos? ')' ';'
  ;

// =====
// TOKENS LÉXICOS
// =====

ID
  : [a-zA-Z_][a-zA-Z0-9_]*
  ;

NUMERO
  : [0-9]+
  ;

TEXTO
  : '"' (~["\r\n])* '"'
  ;

// Ignorar espacios y saltos
WS
  : [ \t\r\n]+ -> skip
  ;

```